

Drive Music — Automated Test Report

Test runner: Vitest 4.1.10 Run date: 2026-07-28 Result:  42 / 42 tests passed across 4 test files — 0 failures

Summary

Test file	Tests	Passed	Failed
src/lib/model.test.ts	12	12	0
src/lib/features.test.ts	10	10	0
src/lib/db.test.ts	14	14	0
src/components/PlayerContext.shuffle.test.ts	6	6	0
Total	42	42	0

Total wall-clock run time: 436 ms.

What this suite covers

This is a **new** test suite added to the project — none existed before. It targets the parts of the codebase that are pure logic and can be tested without a browser: the recommendation model, feature engineering, the IndexedDB storage layer, and the weighted-shuffle algorithm. UI components, Drive API calls, and audio playback are not covered here — they depend on live browser APIs (`<audio>` , `fetch` , `canvas`) that would need a much heavier browser-automation setup to test meaningfully, which was out of scope for this pass.

1. src/lib/model.ts — the recommendation model (12 tests)

#	Test	Result
1	createDefaultModel has the expected shape	
2	createDefaultModel randomizes the hidden layer so units aren't symmetric	
3	predict always returns a probability in [0, 1]	
4	predict is exactly 0.5 for a fresh model (zero output layer) regardless of input	
5	trainStep increments trainingEvents and never mutates the input model	
6	trainStep moves the prediction toward label 1 with repeated training	
7	trainStep moves the prediction toward label 0 with repeated training	
8	Learning one feature's association doesn't blow up predictions for an unrelated feature	
9	weightedRandomIndex returns 0 for an empty weights array	
10	weightedRandomIndex always returns a valid index	

11	weightedRandomIndex picks a heavily-weighted index far more often than a near-zero one	✓
12	weightedRandomIndex distributes roughly proportionally to weight for equal weights	✓

What this proves: the 2-layer network's forward pass is numerically sound (output always a valid probability), backprop actually moves predictions in the correct direction toward the training label without corrupting unrelated inputs, and the weighted-random-pick helper used for "shuffle play" starting tracks is correctly weight-proportional rather than uniform — directly verifying the "does it really use the model" question from earlier in this session.

2. `src/lib/features.ts` — feature engineering (10 tests)

#	Test	Result
1	FEATURE_GROUPS sizes sum to FEATURE_SIZE	✓
2	FEATURE_SIZE is 47 given the current bucket configuration	✓
3	extractFeatures returns a vector of exactly FEATURE_SIZE length	✓
4	Sets the bias feature (index 0) to 1	✓
5	Is one-hot per group — exactly 6 ones total	✓
6	Is deterministic — same inputs produce the identical vector	✓
7	Buckets night/morning/afternoon/evening hours distinctly at the boundaries	✓
8	Buckets weekends separately from weekdays	✓
9	Falls back gracefully to unknown-artist/unknown-album buckets when metadata is missing	✓
10	Still produces a valid vector when the file id is an empty string	✓

What this proves: the hashing scheme is deterministic and collision-tolerant enough to always produce a well-formed one-hot vector, the time/weekday bucket boundaries are exactly where they're documented to be, and missing metadata degrades gracefully instead of throwing.

3. `src/lib/db.ts` — IndexedDB storage layer (14 tests, via `fake-indexeddb`)

#	Test	Result
1	Tracks: round-trips a cached track and reports it as cached	✓
2	Tracks: lists cached tracks sorted by most-recently cached first	✓
3	Tracks: deletes a cached track	✓
4	Playlists: creates a playlist and finds it via <code>listPlaylists</code>	✓
5	Playlists: adds and removes tracks, de-duplicating on add	✓

6	Playlists: renames a playlist	✓
7	Playlists: deletes a playlist	✓
8	Recent sources: upserts a recent source and lists it back	✓
9	Recent sources: re-recording the same id updates in place rather than duplicating	✓
10	Model: <code>loadModel</code> returns a usable default when nothing has been saved	✓
11	Model: round-trips a saved model	✓
12	Model: discards an incompatible (old-architecture) saved model rather than crashing	✓
13	Model events: records an event and lists it back, most recent first	✓
14	Model events: prunes the oldest events once the store exceeds 500 rows	✓

What this proves: every CRUD path the app relies on for offline caching, playlists (including the auto-managed "Favorites" playlist's underlying operations), recent-play history, and the model/training-log persistence behaves correctly — including the two trickiest edge cases: the model-shape migration guard added when the model architecture changed, and the 500-row cap on the training-event log actually pruning the oldest rows rather than growing unbounded.

4. `src/components/PlayerContext.tsx` — weighted shuffle algorithm (6 tests)

#	Test	Result
1	Always returns a full permutation of <code>[0, length)</code>	✓
2	Always pins the given index first	✓
3	Handles a single-track queue (length 1)	✓
4	Degrades to plain uniform shuffle when all weights are equal	✓
5	Biases a heavily-weighted non-pinned track toward earlier positions	✓
6	Falls back to weight 0.5 for an out-of-bounds/undefined weight entry	✓

What this proves: the core correctness guarantee of the shuffle system — "every track plays once before any repeat" — holds under randomized trials, the currently-playing track is always preserved at the front of a freshly dealt order, and a track's weight measurably shifts its expected position rather than being cosmetic. This is a direct regression test for the "Up Next doesn't update" bug fixed earlier in this session, and for the "is shuffle really using the model" question.

How to re-run this suite

```
npm test          # runs `vitest run` once, non-interactively
npx vitest        # interactive watch mode
```

Test files live alongside the code they test (`*.test.ts`), configured via `vitest.config.ts` at the project root.